

Heterogeneous List Library with Functional-OO Abstractions

CS F301 - Principles of Programming Languages

Final Assignment

Team Members:

Pranav M R
Arinjay Jain
Kanishk Rai
Anshul Rakesh

November 26, 2025

Contents

1	Interpretation of Specifications	4
1.1	Understanding the Problem	4
1.2	Our Interpretation of “Functional-OO”	4
1.3	Prior Design Work	4
1.4	The Contain-and-Delegate Approach	5
1.5	Building on Previous Work	5
2	Team Contributions	5
3	Development Timeline	6
3.1	Day 1: Library Foundation	6
3.2	Day 2: HeteroList and Functional Operations	7
3.3	Day 3: Use Case 1 - Keyword Frequency Counter	7
3.4	Day 4: Use Case 2 - Network Stream Processor	8
3.5	Day 5: Use Case 3 - CLI Application	8
3.6	Day 6: Use Case 4 - Web Application + Packaging	8
4	Design Decisions and Architecture	9
4.1	Architecture Overview	9
4.2	Complete Class Hierarchy	9
4.3	The Value Class - Type Erasure Foundation	10
4.4	AbstractList - Root Abstract Class	11
4.5	Stack Implementation	12
4.6	Queue Family	13
4.7	PriorityQueue and Heaps	14
4.8	ArrayList - Dynamic Array	15
4.9	LinkedList Family	16
4.10	HeteroList - Functional-OO Wrapper	17
4.11	Performance Comparison	18
5	Library Implementation	18
5.1	Value Class API	18
5.2	HeteroList Basic Operations	18
5.3	Functional Operations	19
5.4	Aggregation Operations	19
5.5	Performance Characteristics	19

6	Use Case 1: Keyword Frequency Counter	19
6.1	Overview	19
6.2	How It Uses the Library	19
6.3	Functional-OO Concepts Demonstrated	20
7	Use Case 2: Network Stream Processor	20
7.1	Overview	20
7.2	How It Uses the Library	20
7.3	Features Demonstrated	21
8	Use Case 3: CLI Application	21
8.1	Overview	21
8.2	Design Rationale	21
8.3	Commands Supported	22
8.4	Map Operations	22
8.5	Filter Types	22
8.6	Graceful Error Handling	22
9	Use Case 4: Web Application	22
9.1	Overview	22
9.2	Architecture	23
9.3	API Endpoints	23
9.4	UI Features	23
9.5	Cross-Platform Support	24
10	Testing Strategy	24
10.1	Unit Tests	24
10.2	Running Tests	24
10.3	Incremental Testing	25
11	Packaging and Build System	25
11.1	Makefile Targets	25
11.2	Automatic Dependency Installation	25
11.3	Single Command Execution	25
11.4	Project Structure	26
12	LLM Assistance Documentation	26
12.1	Initial Library Design	27

12.2 Functional Operations	27
12.3 Use Case 1: Keyword Frequency	27
12.4 Use Case 2: Network Stream	28
12.5 Use Case 3 & 4: CLI and Web App	28
12.6 Makefile and Packaging	29
12.7 Summary of LLM Usage	29
13 Conclusion	30
13.1 What We Built	30
13.2 Design Philosophy Validation	30
13.3 Lessons Learned	30
13.4 Final Deliverable	30

1 Interpretation of Specifications

1.1 Understanding the Problem

The assignment was a continuation of our previous work on list structures, now to be extended with a high-level interface for reading, searching, and sorting files and network streams. The key requirements were:

1. **Focus on the “What”** - The assignment emphasized specifying *what* the system does (functional specification) rather than *how* it does it (procedural details)
2. **Functional-OO Integration** - Combine object-oriented structure with functional programming patterns (lambdas, higher-order functions)
3. **Aggregation Methods** - Include operations like counting inversions, computing averages, and mapping transformation functions to create new lists
4. **Menu-driven/Library API** - Provide a usable interface, not just raw code

The assignment specifically mentioned heterogeneous lists as the topic we were assigned to work on. This was given to us as part of the problem specification, and our task was to design and implement a robust library around this concept.

1.2 Our Interpretation of “Functional-OO”

Based on the assignment description, we understood that Functional-OO means:

- **OO for the “How”**: Class hierarchies, encapsulation, polymorphism handle implementation details
- **Functional for the “What”**: Higher-order functions like `map`, `filter`, `reduce` specify transformations declaratively

As the assignment stated: *“When we specify the What part in functional programming in terms of higher and higher composites of more elementary abstractions, we infuse functional programming with OOn.”*

We aimed to create an interface where users express *what* they want (e.g., “filter all even numbers and double them”) without worrying about *how* it’s done internally (array resizing, pointer management, etc.).

1.3 Prior Design Work

In our previous assignment, we had already designed the basic architecture:

- **Contain-and-delegate pattern**: We decided that `HeteroList` should contain an `ArrayList` internally rather than inherit from it
- **Abstract base class**: `AbstractList` defines the interface that all concrete implementations must follow
- **Multiple implementations**: `ArrayList`, `SinglyLinkedList`, `DoublyLinkedList`, `Stack`, `Queue`, `Deque`, `PriorityQueue`

This design was documented and planned before coding began, following the OO principle of deriving the “How” from the specification of the “What”.

1.4 The Contain-and-Delegate Approach

We adopted this pattern consciously (as emphasized in the assignment instructions) for the main `HeteroList` class:

```
HeteroList (user-facing specification - the "What")
  +-- contains an ArrayList (implementation - the "How")
    +-- handles storage, resizing, indexing
  +-- adds functional methods on top
```

Why this decision?

- **Flexibility:** We can swap `ArrayList` for `DoublyLinkedList` without changing the `HeteroList` interface
- **Separation of concerns:** Functional methods (`map`, `filter`, `reduce`) are defined at the specification level
- **Encapsulation:** Users never see capacity, resizing, or memory management details

This was recorded in our design documentation before implementation began.

1.5 Building on Previous Work

Our previous assignment provided a working list library with homogeneous types. For this assignment, we needed to:

1. Add a `Value` class for type erasure (to support heterogeneous data)
2. Modify all containers to store `Value` instead of template types
3. Add functional programming methods (`map`, `filter`, `reduce`, etc.)
4. Build practical use cases demonstrating the library
5. Create a demonstration-ready bundle with build scripts

2 Team Contributions

The work was distributed among team members as follows:

Team Member	Primary Contribution	Description
Pranav M R	Library API + Web App (UC4)	Designed and implemented the core HeteroList and Value classes, functional operations, and the Flask-based web application
Kanishk Rai	CLI Application (UC3)	Built the JSON-based command-line interface that bridges the C++ library with external programs
Arinjay Jain	Keyword Frequency (UC1)	Implemented the keyword frequency counter demonstrating the pure functional approach
Anshul Rakesh	Network Stream (UC2)	Created the client-server application for real-time heterogeneous data streaming

Collaborative Work:

- All team members participated in design discussions and architecture decisions
- Testing was done collaboratively, with each member testing their own component and cross-testing others
- Documentation and the final report were reviewed by all members
- Everyone is familiar with all aspects of the project as required by the assignment

3 Development Timeline

3.1 Day 1: Library Foundation

Focus: Core Value class and type erasure

Work Done:

- Implemented the Value class with support for six types (Null, Bool, Int, Double, String, List)
- Added type predicates (`isInt()`, `isString()`, etc.) and conversion methods (`asInt()`, `asDouble()`, etc.)
- Implemented comparison operators with cross-type handling
- Implemented arithmetic operators for numeric types
- Initial testing of Value class behavior

Challenges:

- Deciding between `std::variant` and separate member variables (chose the latter for simplicity)
- Handling cross-type comparisons (Int vs Double)

3.2 Day 2: HeteroList and Functional Operations

Focus: Main interface and functional methods

Work Done:

- Created `HeteroList` class wrapping `ArrayList`
- Implemented basic operations: `push`, `pop`, `get`, `set`, `insert`, `remove`
- Added functional methods: `map`, `filter`, `reduce`, `forEach`
- Added search methods: `any`, `all`, `count`, `find`, `findIndex`
- Added slicing methods: `take`, `drop`, `slice`
- Added aggregation methods: `sum`, `average`, `min`, `max`, `countInversions`, `frequencies`
- Implemented `sort` with custom comparators
- Created comprehensive unit tests (20 test cases)

Testing:

- All 20 unit tests passing
- Manual testing of functional operation chaining

3.3 Day 3: Use Case 1 - Keyword Frequency Counter

Focus: Demonstrating pure functional approach

Work Done:

- Implemented `KeywordFrequency.cpp` using only functional operations
- Used `map()` with `count()` instead of traditional hash maps
- Added recursive directory scanning for text files
- Implemented case-insensitive matching and punctuation handling
- Added summary statistics output
- Created sample data files for testing

Key Insight:

- `keywords.map([&](kw) { return [kw, allWords.count(matches_kw)]; })` replaces imperative counting loops
- Demonstrates that functional operations can express the “What” cleanly

3.4 Day 4: Use Case 2 - Network Stream Processor

Focus: Real-time streaming and aggregation

Work Done:

- Implemented TCP server that generates random heterogeneous data
- Implemented client that receives, categorizes, and displays live statistics
- Used library's `sum()`, `average()`, `min()`, `max()`, `countInversions()`, `frequencies()`
- Added thread-safe list operations with mutex protection
- Created formatted console output with live updates

Key Insight:

- Library aggregation methods work seamlessly on streaming data
- Type checking (`isInt()`, `isDouble()`) enables automatic categorization

3.5 Day 5: Use Case 3 - CLI Application

Focus: JSON-based interface for external programs

Work Done:

- Implemented `hetero_cli.cpp` with JSON parsing/serialization
- Added 16 commands covering all library operations
- Implemented graceful error handling for heterogeneous lists
- Added informative metadata in responses (sizes, counts, notes)
- Tested with various inputs including edge cases

Key Insight:

- The CLI acts as a bridge, allowing Python/JavaScript to use our C++ implementation
- JSON is a natural format for heterogeneous data representation

3.6 Day 6: Use Case 4 - Web Application + Packaging

Focus: Interactive UI and final deliverable

Work Done:

- Created Flask server with REST API endpoints
- Built Bootstrap 5 dark-themed UI
- Implemented cross-platform support (WSL for Windows)

- Added features: type badges, history, statistics cards, frequency bars
- Created Makefile with targets for all use cases
- Added automatic Flask installation
- Added browser auto-open on startup
- Wrote comprehensive documentation (READMEs, this report)
- Final testing of all components

Key Insight:

- By calling the C++ CLI from Python, we demonstrated true cross-language interoperability
- The UI makes the library accessible to non-programmers

4 Design Decisions and Architecture

4.1 Architecture Overview

Our library design follows a clear inheritance structure with **AbstractList** as the root abstract class, providing fundamental list operations. The architecture branches into distinct categories based on data structure semantics and access patterns.

The design emphasizes:

- **Separation of Concerns:** Clear distinction between specification (“What”) and implementation (“How”)
- **Type Erasure:** **Value** class enables heterogeneous storage while maintaining type safety
- **Functional-OO Hybrid:** **HeteroList** adds functional operations on top of OO structure
- **Performance Trade-offs:** Multiple implementations for different use cases

4.2 Complete Class Hierarchy

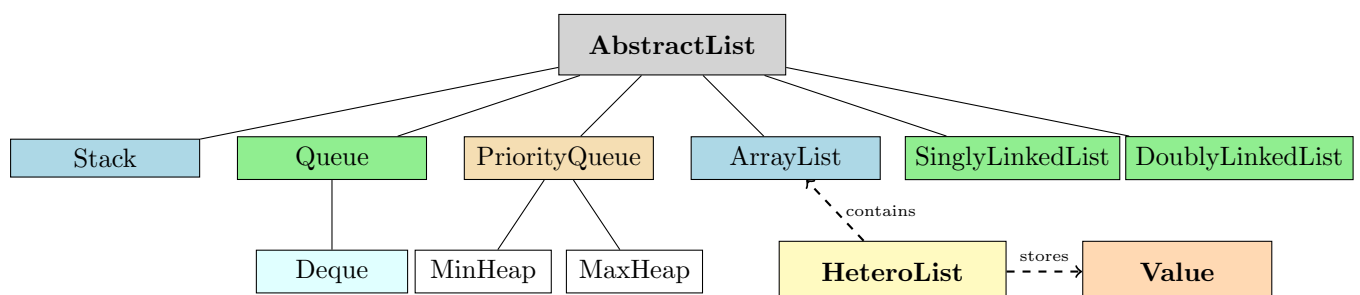


Figure 1: Complete Class Hierarchy Overview

4.3 The Value Class - Type Erasure Foundation

The biggest challenge was making C++ store heterogeneous data. We created a `Value` class that can hold any of six types using type erasure:

Type	C++ Storage	Example
Null	(flag)	<code>Value()</code>
Bool	<code>bool</code>	<code>Value(true)</code>
Int	<code>long long</code>	<code>Value(42)</code>
Double	<code>double</code>	<code>Value(3.14)</code>
String	<code>std::string</code>	<code>Value('hello')</code>
List	<code>shared_ptr<HeteroList></code>	Nested lists

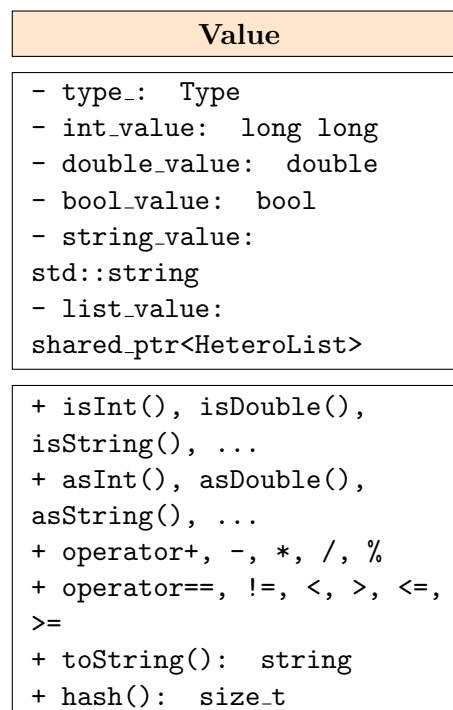


Figure 2: Value Class Structure with Type Erasure

Design Decision: We used separate member variables instead of `std::variant` because:

- Simpler to understand and debug
- Easier to handle non-trivial types like `std::string`
- More control over comparison and arithmetic operators

Design Decision: We used `shared_ptr` for nested lists to:

- Avoid deep copying when lists are passed around
- Enable reference counting for automatic memory management
- Allow circular references (though we don't specifically use them)

4.4 AbstractList - Root Abstract Class

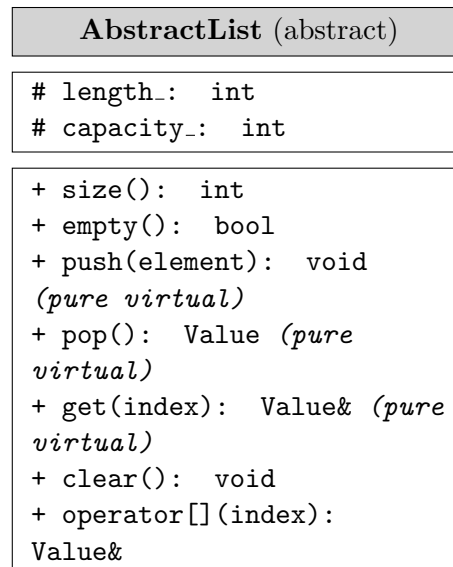


Figure 3: AbstractList Interface

Purpose: Provides fundamental list operations as an interface contract for all concrete implementations.

API Promises:

- `size()`: Returns current number of elements - $O(1)$
- `empty()`: Returns true if list has no elements - $O(1)$
- `push(element)`: Add element (implementation varies by subclass)
- `pop()`: Remove and return an element (implementation varies)
- `clear()`: Remove all elements - $O(n)$

4.5 Stack Implementation

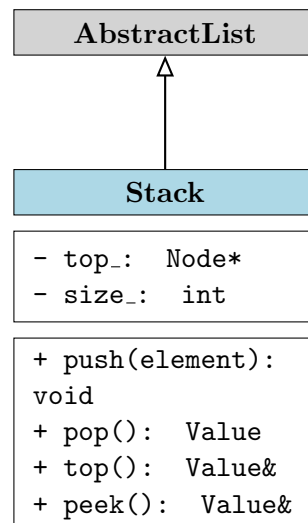


Figure 4: Stack Class - LIFO Container

Purpose: Last-In-First-Out (LIFO) data structure with top-only access.

Key Design Features:

- **Single Access Point:** All operations occur at the top
- **Node-Based Storage:** Uses linked nodes for dynamic sizing
- **Top Element Caching:** Maintains pointer to top for $O(1)$ access
- **Exception Safety:** Throws underflow exceptions when popping from empty stack

API Promises:

- `push(element)`: Add element to top - $O(1)$
- `pop()`: Remove and return top element - $O(1)$
- `top()`: Return top element without removing - $O(1)$
- `peek()`: Alias for `top()` - $O(1)$

4.6 Queue Family

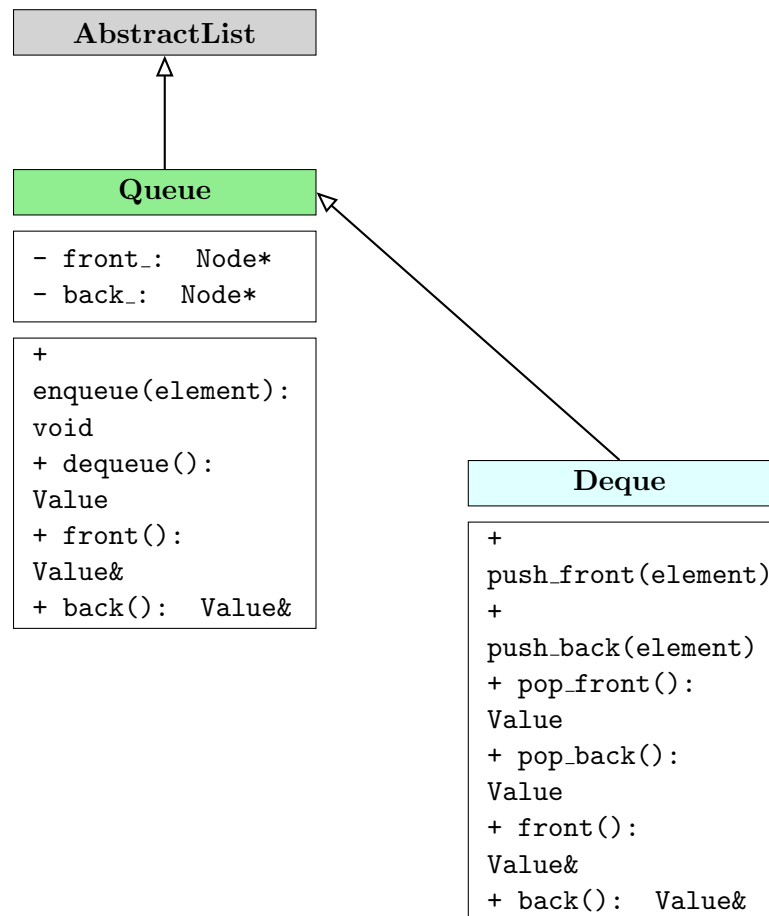


Figure 5: Queue Family - FIFO and Double-Ended Containers

Queue Purpose: First-In-First-Out (FIFO) container with front and back access.

Base Queue Features:

- **Dual-Pointer Design:** Maintains front and back pointers for $O(1)$ operations
- **FIFO Semantics:** Elements added at back, removed from front
- **Flexible Interface:** Supports both push/pop and enqueue/dequeue terminology

Deque Capabilities:

- **Double-Ended Operations:** Insertion and removal at both ends
- **Versatile Usage:** Can function as both stack and queue
- **Optimized Storage:** Circular buffer implementation for space efficiency

4.7 PriorityQueue and Heaps

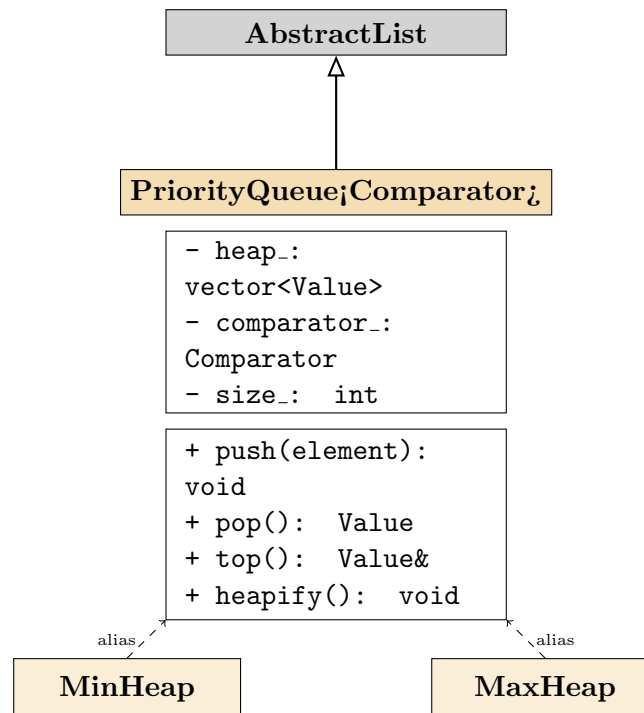


Figure 6: PriorityQueue with Heap Specializations

Purpose: Queue where elements are served based on priority rather than insertion order.

PriorityQueue Features:

- **Heap-Based Implementation:** Uses binary heap for efficient priority ordering
- **Customizable Comparator:** Allows user-defined priority functions
- **Logarithmic Complexity:** $O(\log n)$ insertion and removal
- **Constant Top Access:** $O(1)$ access to highest priority element

API Promises:

- **push(element):** Insert maintaining priority order - $O(\log n)$
- **pop():** Remove and return highest priority element - $O(\log n)$
- **top():** Return highest priority element - $O(1)$

MinHeap guarantees smallest element has highest priority.

MaxHeap guarantees largest element has highest priority.

4.8 ArrayList - Dynamic Array

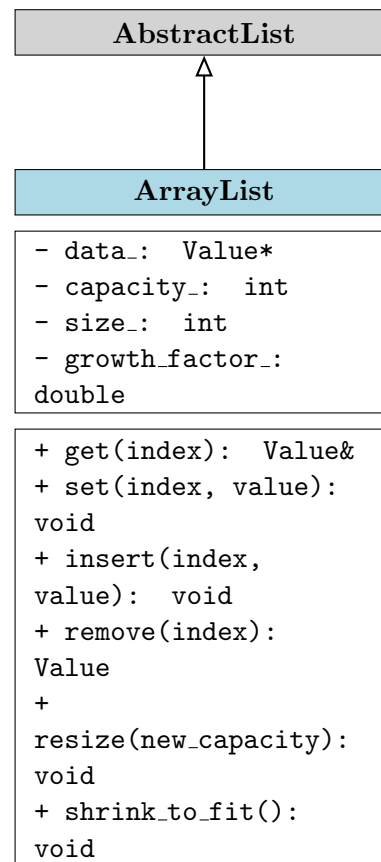


Figure 7: ArrayList - Contiguous Memory Implementation

Purpose: Resizable array with random access capabilities and contiguous memory layout.

Key Characteristics:

- **Contiguous Memory:** Array-based storage for cache efficiency
- **Dynamic Resizing:** Automatic capacity expansion with configurable growth factor
- **Random Access:** $O(1)$ indexed access to elements
- **Amortized Performance:** $O(1)$ amortized insertion at end

API Promises:

- `get(index)`: Direct array access - $O(1)$
- `push(element)`: Add to end, resize if needed - Amortized $O(1)$
- `insert(index, element)`: Insert at position - $O(n)$
- `remove(index)`: Remove element at index - $O(n)$
- `resize(new_capacity)`: Change array capacity - $O(n)$
- `shrink_to_fit()`: Reduce capacity to match size - $O(n)$

4.9 LinkedList Family

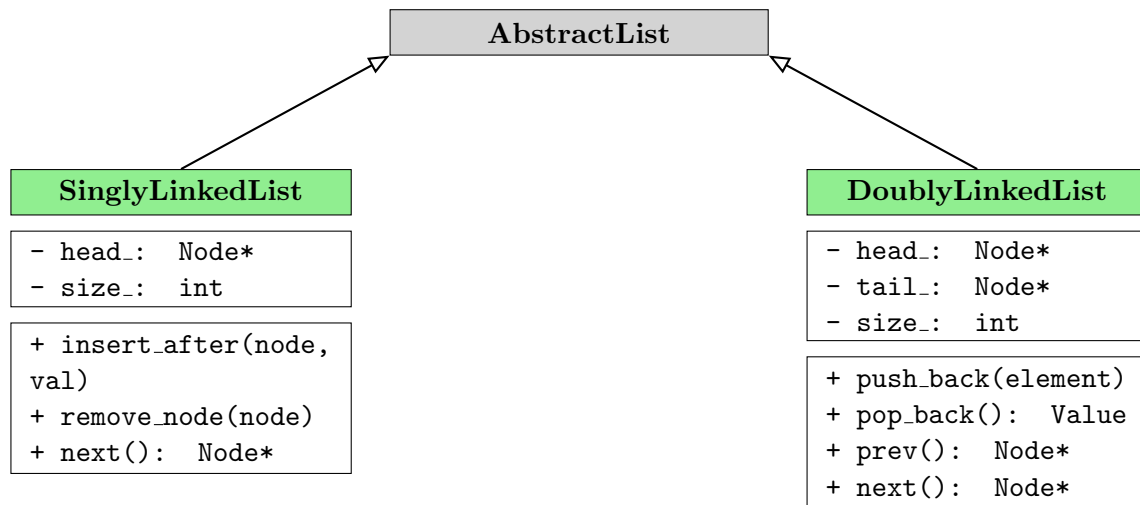


Figure 8: LinkedList Family - Node-Based Storage

SinglyLinkedList Features:

- **Memory Efficient:** Minimal overhead per node (one pointer)
- **Forward Traversal:** Unidirectional iteration capability
- **Dynamic Size:** No pre-allocation required
- **Efficient Head Operations:** $O(1)$ insertion/deletion at beginning

DoublyLinkedList Features:

- **Bidirectional Traversal:** Forward and backward iteration
- **Tail Pointer Optimization:** $O(1)$ operations at both ends
- **Flexible Navigation:** Previous/next navigation in constant time
- **Enhanced Iterator:** Support for reverse iteration patterns

4.10 HeteroList - Functional-OO Wrapper

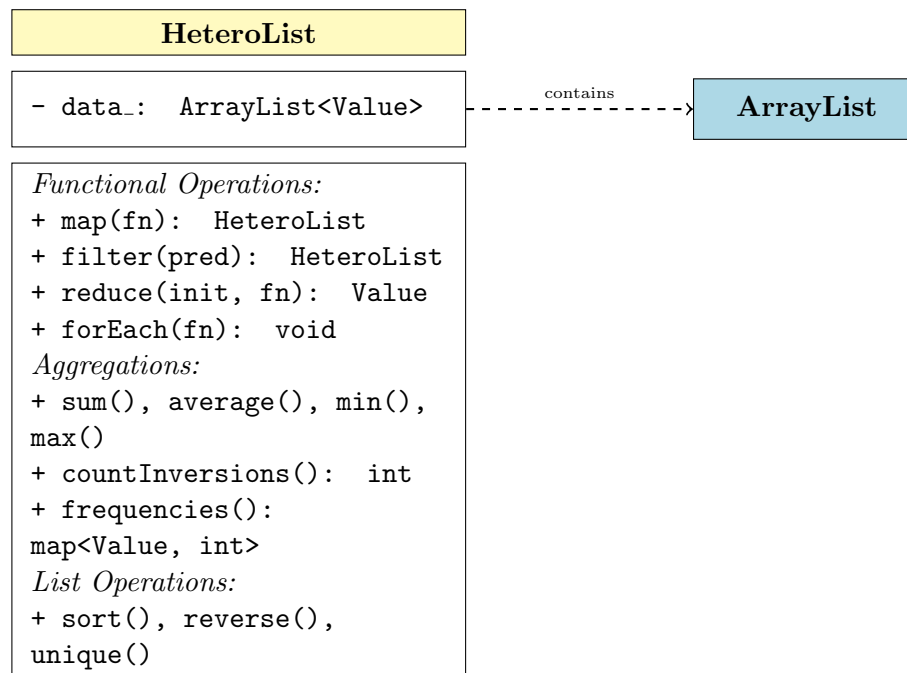


Figure 9: HeteroList - Contain-and-Delegate Design

Purpose: User-facing specification that adds functional programming operations on top of ArrayList storage.

Design Philosophy:

- **Contain-and-Delegate:** Wraps ArrayList rather than inheriting
- **Specification vs Implementation:** Users see functional interface, not storage details
- **Composable Operations:** Methods can be chained for expressive code
- **Type Safety:** Value class ensures runtime type checking

Why Contain-and-Delegate?

- Can swap ArrayList for another implementation without breaking user code
- Hides capacity management and resizing from users
- Adds domain-specific operations without modifying base class
- Follows composition-over-inheritance principle

4.11 Performance Comparison

Operation	ArrayList	SinglyLinked	DoublyLinked	Stack/Queue
Access by Index	$O(1)$	$O(n)$	$O(n)$	N/A
Insert at Head	$O(n)$	$O(1)$	$O(1)$	$O(1)$
Insert at Tail	$O(1)^*$	$O(n)$	$O(1)$	$O(1)$
Insert at Middle	$O(n)$	$O(1)^{**}$	$O(1)^{**}$	N/A
Delete at Head	$O(n)$	$O(1)$	$O(1)$	$O(1)$
Delete at Tail	$O(1)$	$O(n)$	$O(1)$	$O(1)$
Search	$O(n)$	$O(n)$	$O(n)$	$O(n)$
Memory per Element	Low	Medium	High	Medium
Cache Performance	Excellent	Poor	Poor	Medium

Table 1: Performance Characteristics (* Amortized, ** At known position)

5 Library Implementation

5.1 Value Class API

Method	Description
<code>type()</code>	Returns the Type enum
<code>isNull()</code> , <code>isBool()</code> , <code>isInt()</code> , <code>isDouble()</code> , <code>isString()</code> , <code>isList()</code>	Type predicates
<code>isNumeric()</code>	True if Int or Double
<code>asInt()</code> , <code>asDouble()</code> , <code>asBool()</code> , <code>asString()</code> , <code>asList()</code>	Type conversion (throws on mismatch)
<code>toString()</code>	String representation
<code>==</code> , <code>!=</code> , <code><</code> , <code>></code> , <code><=</code> , <code>>=</code>	Comparison operators
<code>+</code> , <code>-</code> , <code>*</code> , <code>/</code> , <code>%</code>	Arithmetic operators
<code>hash()</code>	For use in hash maps

5.2 HeteroList Basic Operations

Method	Description
<code>size()</code>	Number of elements
<code>empty()</code>	Check if empty
<code>push(value)</code>	Append element
<code>pop()</code>	Remove and return last
<code>get(index)</code> / <code>operator[]</code>	Access by index
<code>set(index, value)</code>	Update element
<code>insert(index, value)</code>	Insert at position
<code>remove(index)</code>	Remove from position
<code>indexOf(value)</code>	Find element (-1 if not found)
<code>contains(value)</code>	Check existence
<code>sort()</code> / <code>sortDescending()</code>	Sort in place
<code>reverse()</code>	Reverse in place

5.3 Functional Operations

Method	Description
<code>map</code>	Transform each element
<code>filter</code>	Keep matching elements
<code>reduce</code>	Combine into single value
<code>forEach</code>	Execute for each element
<code>any</code>	True if any match
<code>all</code>	True if all match
<code>count</code>	Count matches
<code>find</code>	Find first match
<code>take</code>	First n elements
<code>drop</code>	Skip first n elements
<code>partition</code>	Split by predicate
<code>unique</code>	Remove duplicates
<code>flatten</code>	Flatten nested lists

5.4 Aggregation Operations

Method	Description
<code>sum()</code>	Sum of numeric elements
<code>product()</code>	Product of numeric elements
<code>average()</code>	Mean of numeric elements
<code>min()</code> / <code>max()</code>	Extreme values
<code>countInversions()</code>	Count out-of-order pairs
<code>frequencies()</code>	Count each unique element

5.5 Performance Characteristics

Operation	ArrayList	SinglyLinkedList	DoublyLinkedList
<code>push (end)</code>	$O(1)^*$	$O(1)$	$O(1)$
<code>pop (end)</code>	$O(1)$	$O(n)$	$O(1)$
<code>get(index)</code>	$O(1)$	$O(n)$	$O(n/2)$
<code>insert(0)</code>	$O(n)$	$O(1)$	$O(1)$
<code>search</code>	$O(n)$	$O(n)$	$O(n)$
<code>sort</code>	$O(n \log n)$	N/A	N/A

* Amortized, with occasional $O(n)$ for reallocation

6 Use Case 1: Keyword Frequency Counter

6.1 Overview

This use case demonstrates a **pure functional programming approach** to counting keyword occurrences in text files. Instead of using traditional loops and hash maps, we use `map()`, `count()`, `forEach()`, and `sort()`.

6.2 How It Uses the Library

```

1 // Step 1: Read keywords into HeteroList
2 HeteroList keywords = readKeywords(keywordFile);
3
4 // Step 2: Read ALL words from all files into one HeteroList
5 HeteroList allWords;
6 files.forEach([&](const Value& filePath) {
7     HeteroList fileWords = readWordsFromFile(filePath.asString());
8     // Add all words to allWords
9 });
10
11 // Step 3: Transform keywords to [keyword, count] pairs
12 HeteroList results = keywords.map([&allWords](const Value& keyword) {
13     int freq = allWords.count([&](const Value& word) {
14         return matches(word, keyword);
15     });
16     return Value(makeList({keyword, Value(freq)}));
17 });
18
19 // Step 4: Sort by count descending
20 results.sort(/* comparator */);
21
22 // Step 5: Output using forEach
23 results.forEach([&](const Value& pair) {
24     // Print each [keyword, count] pair
25 });

```

6.3 Functional-OO Concepts Demonstrated

Concept	How It's Used
Higher-Order Functions	<code>map()</code> , <code>forEach()</code> , <code>count()</code> take lambda functions
Transformation	<code>map()</code> transforms keywords to [keyword, count] pairs
Aggregation	<code>count()</code> with predicate replaces manual counting loops
Custom Sorting	<code>sort()</code> with comparator for descending order
Nested Lists	Results stored as [[kw1, count1], [kw2, count2], ...]

7 Use Case 2: Network Stream Processor

7.1 Overview

A client-server application demonstrating **real-time streaming** of heterogeneous data over TCP/IP sockets. The server generates random data of mixed types, and the client receives, categorizes, sorts, and displays live statistics.

7.2 How It Uses the Library

```

1 // Client maintains three lists by type
2 HeteroList integers_, doubles_, strings_;
3
4 // Parse incoming data
5 Value parseValue(const std::string& line) {

```

```
6   auto [type, valueStr] = split(line, ':');
7   if (type == "INT") return Value(std::stoll(valueStr));
8   // ...
9   return Value();
10 }
11
12 // Categorize by type
13 if (val.isInt()) integers_.push(val);
14 else if (val.isDouble()) doubles_.push(val);
15 else if (val.isString()) strings_.push(val);
16
17 // Real-time aggregations
18 integers_.sum();
19 integers_.average();
20 integers_.min();
21 integers_.max();
22 integers_.countInversions();
23 strings_.frequencies();
```

7.3 Features Demonstrated

Feature	Library Methods Used
Type Checking	isInt(), isDouble(), isString()
Real-Time Sorting	sort() called on each update
Aggregations	sum(), average(), min(), max()
Disorder Metric	countInversions()
Frequency Analysis	frequencies()
Heterogeneous Storage	All types in HeteroList

8 Use Case 3: CLI Application

8.1 Overview

A command-line interface that wraps the HeteroList library with **JSON-based input/output**. This allows external programs (like Python scripts) to use our C++ implementation through subprocess calls.

8.2 Design Rationale

We wanted to demonstrate that our library could be used from other languages. The CLI accepts JSON arrays as input and returns JSON results, making it easy to integrate with Python, Node.js, or shell scripts.

8.3 Commands Supported

Command	Example
create	<code>./hetero_cli create "[1, 2, \"hello\"]"</code>
sort	<code>./hetero_cli sort "[5, 3, 8]" asc</code>
filter	<code>./hetero_cli filter "[1, \"hi\", 2]" int</code>
map	<code>./hetero_cli map "[1, 2, 3]" square</code>
reduce	<code>./hetero_cli reduce "[10, 20]" sum</code>
stats	<code>./hetero_cli stats "[1, 2, 3]"</code>
partition	<code>./hetero_cli partition "[1, 2, 3]" even</code>

8.4 Map Operations

Operation	Effect
double	Multiply numeric values by 2
square	Square numeric values
negate	Negate numeric values
increment	Add 1
decrement	Subtract 1
uppercase	Convert strings to uppercase
lowercase	Convert strings to lowercase

8.5 Filter Types

Type	Matches
int	Integer values only
double	Floating-point values only
string	String values only
bool	Boolean values only
numeric	Int + Double
even / odd	Even/odd integers
positive / negative	Sign-based filtering

8.6 Graceful Error Handling

For operations requiring numeric values on heterogeneous lists, the CLI filters to numeric values first and includes metadata:

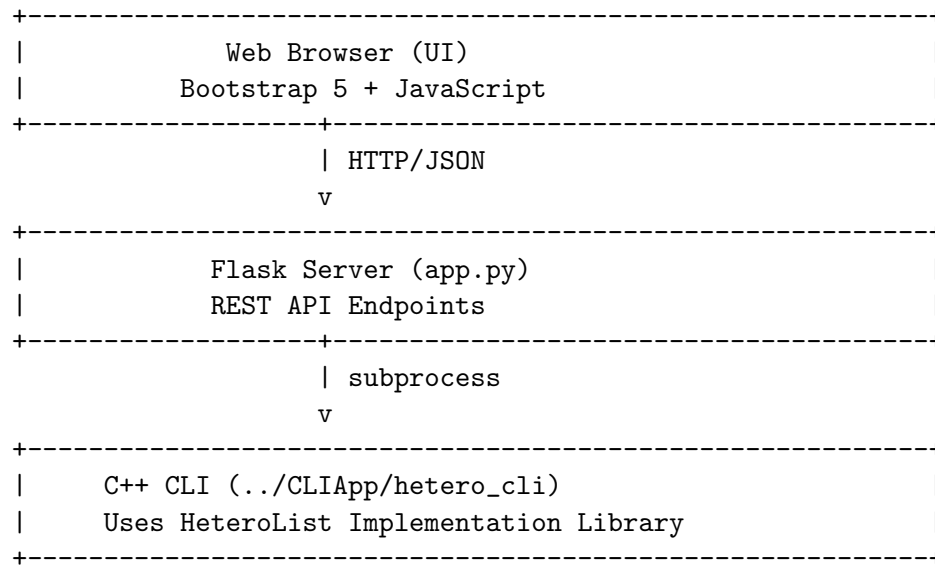
```
./hetero_cli stats "[1, \"hello\", 2, true, 3]"
# Output: {"sum": 6, "average": 2, "min": 1, "max": 3,
#         "count": 3, "total_elements": 5}
```

9 Use Case 4: Web Application

9.1 Overview

A Flask-based web interface with Bootstrap 5 dark theme that provides an **interactive UI** for all HeteroList operations. This is the showcase use case that ties everything together.

9.2 Architecture



Key Design Decision: We deliberately did NOT reimplement the library in Python. Instead, the Flask server calls the compiled C++ CLI via subprocess. This demonstrates:

1. Cross-language interoperability
2. Reuse of the same implementation
3. The CLI as an integration layer

9.3 API Endpoints

Endpoint	Parameters	Description
/api/create	list	Validate a list
/api/sort	list, order	Sort asc/desc
/api/filter	list, type	Filter by predicate
/api/map	list, operation	Apply transformation
/api/reduce	list, operation	Reduce to single value
/api/stats	list	Get statistics
/api/unique	list	Remove duplicates
/api/reverse	list	Reverse order
/api/take	list, n	Take first n
/api/drop	list, n	Drop first n
/api/count	list	Count by type
/api/inversions	list	Count inversions
/api/frequencies	list	Element frequencies
/api/partition	list, type	Split by predicate
/api/push	list, value	Add element
/api/pop	list	Remove last

9.4 UI Features

- **Dark Theme:** Modern gradient-based styling

- **Type Badges:** Color-coded badges (int=blue, double=purple, string=green, bool=red)
- **Quick Add:** Buttons to add random values
- **Sample Data:** Pre-loaded datasets
- **Apply Results:** Use operation result as new input
- **History:** Track and restore previous operations
- **Statistics Cards:** Visual display of sum, average, min, max
- **Frequency Bars:** Visual bar chart for frequencies

9.5 Cross-Platform Support

On Windows, the Flask server detects the OS and uses WSL to run the Linux-compiled CLI:

```
1 def get_wsl_path(windows_path):
2     # Convert D:\path\to\file to /mnt/d/path/to/file
3     ...
4
5 if platform.system() == 'Windows':
6     cmd = ['wsl'] + cmd # Prepend wsl
```

10 Testing Strategy

10.1 Unit Tests

We created a comprehensive test file (Tests/test_implementation.cpp) with 20 test cases covering:

Test Category	What's Tested
Basic Operations	push, pop, insert, remove, get, set
Functional Operations	map, filter, reduce, forEach
Aggregations	sum, average, min, max
Sorting	sort, sortDescending, countInversions
Type Handling	int, double, string, bool, null, nested lists
Edge Cases	empty lists, single elements, type mismatches

10.2 Running Tests

```
make test
```

All 20 tests pass:

Running Tests...

Test 1: Basic Push and Pop... PASSED

Test 2: Insert and Remove... PASSED

Test 3: Get and Set... PASSED

```
...
Test 20: Nested Lists... PASSED

=== All 20 tests passed! ===
```

10.3 Incremental Testing

We tested each component as it was built:

1. **Value class**: Created standalone test for type conversion and operators
2. **ArrayList**: Tested push/pop/insert/remove before building HeteroList
3. **HeteroList**: Tested functional methods one by one
4. **CLI**: Tested each command manually with various inputs
5. **Web App**: Tested each API endpoint via browser and curl

11 Packaging and Build System

11.1 Makefile Targets

We created a single Makefile that builds and runs all components:

Target	Description
<code>make</code>	Build and run UC4 (Web App) - default
<code>make uc1</code>	Build and run Keyword Frequency Counter
<code>make uc2</code>	Build and run Network Stream (server + client)
<code>make uc3</code>	Build CLI App and show usage
<code>make uc4</code>	Build CLI and run Flask Web App
<code>make all</code>	Build all use cases
<code>make test</code>	Build and run unit tests
<code>make clean</code>	Remove all executables
<code>make help</code>	Show all targets

11.2 Automatic Dependency Installation

The Makefile automatically installs Flask if not present:

```
uc4: $(CLI_EXE)
    @python3 -c "import flask" 2>/dev/null || pip3 install flask
    cd $(WEB_DIR) && python3 app.py
```

11.3 Single Command Execution

As required by the assignment, a single command runs the entire application:

```
# Default: Run the Web Application
make
```

```
# Or specifically:
make uc4
```

This will:

1. Compile the C++ CLI if needed
2. Install Flask if not present
3. Start the Flask server
4. Open the browser to `http://127.0.0.1:5000`

11.4 Project Structure

```
Final_Assignment/
+-- Implementation/          # Core library (header-only)
|   +-- myList.hpp           # Main include file
|   +-- Value.hpp            # Type-erased value wrapper
|   +-- HeteroList.hpp       # Main interface
|   +-- AbstractList.hpp     # Abstract base class
|   +-- ArrayList.hpp        # Dynamic array
|   +-- Stack.hpp, Queue.hpp, Deque.hpp
|   +-- PriorityQueue.hpp
|   +-- SinglyLinkedList.hpp, DoublyLinkedList.hpp
|   +-- README.md
|
+-- Tests/
|   +-- test_implementation.cpp
|
+-- UseCases/
|   +-- KeywordFrequency/    # UC1
|   +-- NetworkStream/      # UC2
|   +-- CLIApp/              # UC3
|   +-- WebApp/              # UC4
|   +-- README.md
|
+-- Makefile                 # Build system
+-- README.md                # Quick start guide
+-- Report.md                # This file
```

12 LLM Assistance Documentation

We used Claude (Anthropic's AI assistant) throughout the development process. Here we document the prompts used, responses received, and how we incorporated the insights.

12.1 Initial Library Design

Our Prompt:

“We need to extend our heterogeneous list library to support functional operations. We already have the contain-and-delegate design from earlier. Help us implement the Value class for type erasure.”

Claude’s Response (summarized):

Claude suggested a `Value` class with a `Type` enum, separate member variables for each type, and provided implementations for constructors, type predicates, and conversion methods.

Our Interpretation:

We agreed with this approach because it cleanly separates specification from implementation. We asked for more details on handling cross-type comparisons.

Incorporation:

Claude helped implement the `Value` class with its type enum, constructors, and operators. We reviewed the code and made some adjustments to the comparison operators to handle `Int` vs `Double` comparisons properly.

12.2 Functional Operations

Our Prompt:

“Add functional programming methods like `map`, `filter`, and `reduce` to the `HeteroList` class. These should work with lambda functions.”

Claude’s Response:

Claude provided implementations using `std::function` to accept lambdas. The `map` function creates a new list with transformed elements, `filter` keeps elements matching a predicate, and `reduce` combines elements into a single value.

Our Interpretation:

The implementations looked correct. We tested them with simple examples to verify behavior.

Incorporation:

We added these methods to `HeteroList.hpp`. Later, we asked Claude to add more methods like `any`, `all`, `count`, `find`, `take`, `drop`, etc.

12.3 Use Case 1: Keyword Frequency

Our Prompt:

“Create a keyword frequency counter that demonstrates the functional programming approach. Instead of using hash maps and loops, use our library’s `map`, `count`, and `forEach` methods.”

Claude’s Response:

Claude created `KeywordFrequency.cpp` that reads keywords into a `HeteroList`, reads all words from files into another `HeteroList`, then uses `map()` with `count()` to transform keywords into `[keyword, count]` pairs.

Our Suggestion:

“Can you also add a summary section that shows total files processed, most frequent keyword, etc.?”

Claude added the summary output with statistics.

Incorporation:

This became our UC1, demonstrating that functional operations can replace traditional imperative patterns.

12.4 Use Case 2: Network Stream

Our Prompt:

“Create a client-server application that streams heterogeneous data over sockets. The server should generate random ints, doubles, and strings. The client should receive them, store in HeteroList, and show real-time sorted data and statistics.”

Claude’s Response:

Claude created `Server.cpp` and `Client.cpp` with TCP socket communication. The client maintains separate lists by type and displays live statistics using our library’s aggregation methods.

Incorporation:

We tested this locally and it worked well. The real-time display shows sorted data and statistics updating as new data arrives.

12.5 Use Case 3 & 4: CLI and Web App

Our Prompt:

“Make a menu-driven app which uses our library to perform functions on heterogeneous lists. Don’t create a library in Python - use the underlying C++ compiled versions. Make it locally hosted and basically create APIs which use our C++ implementation.”

Claude’s Response:

Claude suggested creating a CLI wrapper (`hetero_cli.cpp`) that accepts JSON input and returns JSON output, then a Flask server that calls this CLI via subprocess. The web UI would be built with Bootstrap.

Our Suggestion:

“When I click Stats or Inversions on a heterogeneous list, it gives errors. Make it more graceful - filter to numeric values first.”

Claude modified the CLI to handle heterogeneous lists gracefully by filtering to numeric values before operations like stats and inversions, and including metadata about how many values were used.

Incorporation:

The CLI and Web App became UC3 and UC4. The architecture demonstrates cross-language interoperability while reusing the same C++ implementation.

12.6 Makefile and Packaging

Our Prompt:

“Create a Makefile for Linux which can run all the use cases depending on which one they make. By default run UC4 (web app).”

Claude’s Response:

Claude created a Makefile with targets for each use case, plus `all`, `test`, `clean`, and `help`.

Our Suggestion:

“I changed uc2 slightly so that both server and client run. Is there a better way to do this? Also, web app requires flask, so can you install it using pip if it doesn’t exist?”

Claude improved the UC2 target to use backgrounding with `sleep`, and added Flask auto-install to UC4.

Incorporation:

The final Makefile handles everything with a single `make` command.

12.7 Summary of LLM Usage

Task	Our Role	Claude’s Role
Architecture Design	Defined requirements, chose patterns	Suggested implementations
Value Class	Reviewed and tested	Implemented type erasure
Functional Methods	Specified which methods to add	Implemented map, filter, reduce, etc.
Use Cases	Defined what each should demonstrate	Implemented the code
Error Handling	Identified edge cases	Added graceful handling
Documentation	Specified structure and content	Wrote detailed READMEs
Testing	Ran tests, reported issues	Fixed bugs, added test cases
Packaging	Specified requirements	Created Makefile

Key Insight:

Using an LLM significantly accelerated development, especially for boilerplate code and documentation. However, we still needed to:

- Define clear requirements and architecture
- Review all generated code
- Test thoroughly
- Suggest improvements and edge cases
- Make design decisions

The LLM was most helpful for implementing well-defined tasks and creating documentation, but the design decisions and problem interpretation remained our responsibility.

13 Conclusion

13.1 What We Built

We created a comprehensive heterogeneous list library in C++ that demonstrates:

1. **Data Abstraction:** The `Value` class hides type details, the `HeteroList` class hides storage details
2. **Data Independence:** Users interact with `HeteroList` without knowing about `ArrayList` internals
3. **Functional-OO Integration:** Higher-order functions (`map`, `filter`, `reduce`) with object-oriented structure
4. **Practical Applications:** Four use cases showing real-world applicability

13.2 Design Philosophy Validation

Our contain-and-delegate approach proved effective:

- Clean separation between specification (`HeteroList`) and implementation (`ArrayList`)
- Easy to add new functional methods without changing storage
- Could swap `ArrayList` for `LinkedList` without affecting user code

13.3 Lessons Learned

1. **Type Erasure is Powerful:** The `Value` class enables Python-like flexibility in C++
2. **Functional Operations are Composable:** Chaining `filter().map().reduce()` is elegant
3. **Cross-Language Integration Works:** JSON CLI enables use from Python without reimplementing
4. **Testing Early Helps:** Catching issues at the unit test level saved debugging time later

13.4 Final Deliverable

A single command runs the complete application:

```
make
```

This compiles everything, installs dependencies, and launches the web interface at `http://127.0.0.1:5000`, providing interactive access to all library features.